

AI Sandbox — Developer Handoff

Current baseline, architecture, security invariants, and continuation rules

1. Current Baseline

Repository: private GitHub repository tanmay3114/ai-sandbox. Current verified baseline commit: bae5f51. Branch: main. Working tree was clean and synchronized with origin/main.

Phases 1–3 are implemented and verified. The existing repository is the source of truth for actual behavior.

2. Instructions for the Next Developer / Coding Agent

- Do not rebuild Phase 1–3.
- Read the PRD and previous phase prompts for context, not as instructions to rerun completed work.
- Inspect the current code and tests before changing anything.
- Start from the next phase specification.
- Avoid unrelated refactors and new infrastructure unless explicitly required.
- Preserve existing API behavior where possible.
- Run the full test suite and Ruff after changes.

3. Critical Architecture

```
Logical Sandbox (PostgreSQL)
|
+-- ExecutionJob 1 -> fresh container -> destroyed
+-- ExecutionJob 2 -> fresh container -> destroyed
+-- ExecutionJob 3 -> fresh container -> destroyed
```

A Sandbox is not a long-lived Docker container. The current platform creates a fresh execution container for each job and removes it afterward. Writable filesystem state is not persisted between jobs.

4. Critical Security Invariants

- network_mode=none
- read_only=true
- user=10001:10001
- cap_drop=ALL
- no-new-privileges=true
- memory=256 MB
- CPU approximately 0.5 core
- PID limit=32
- bounded stdout/stderr
- 32 MB restricted /tmp
- no host mounts

- no Docker socket
- no arbitrary image selection
- mandatory cleanup
- do not touch unrelated Docker containers

5. Environment and Secrets

The repository includes `.env.example` but must not contain the real `.env`. A developer should create a local `.env` from the example and use their own credentials. Never commit passwords, API keys, database credentials, tokens, or private keys.

6. Suggested Workflow

1. Clone the repository
2. Read PRD
3. Read this handoff
4. Read phase prompts
5. Inspect code/tests
6. Implement only the requested phase
7. Run tests
8. Run Ruff
9. Verify security invariants
10. Review git diff
11. Commit
12. Push

7. Handoff Warning

Do not assume that an old prompt overrides the current repository. If a prompt and current implementation appear inconsistent, inspect the code/tests and resolve the difference deliberately before modifying the architecture.